

AgentWorkShop: A Node-Native Integration Framework for LLM Multi-Agent Closed-Loop Control of Heterogeneous Industrial Production Lines

First Author, Second Author, and Third Author

Abstract—Industrial LLM-agent deployments stop at the advisory boundary: published systems diagnose, generate PLC code, or recommend setpoints; to our knowledge, none integrates an agent team into a production line’s own protocol, data, and semantic fabric. We present AgentWorkShop, a configuration-driven integration framework that embeds LLM multi-agent teams natively into the supervisory fabric of heterogeneous production lines: a multi-harness agent runtime (14 interchangeable engines) and an industrial runtime (five protocol drivers with per-node edge acquisition) execute as one process over one data domain. Commissioning is configuration, not code: one plant-scenario identifier re-instantiates the deployment on a second, independently defined line with zero platform code changes (33 seeded out-of-window/range writes intercepted across both scenarios, Wilson 95 % [89.6 %, 100 %]; zero false blocks). Every agent-issued write traverses one governed path—recipe-window interlock, mode-bound approval, readback-verified write, attributable journal—so the accepted-action language is constrained by construction, and the backstop’s automatic rollback is exercised end-to-end on a live breach. All plants are seeded simulators. A governed closed-loop benchmark on a cast-film twin recovers 97.0–97.2 % of the offline grid optimum in two iterations and at most two governed writes per seed under a fixed non-learning controller; four released LLM-agent-team campaigns closed the same loop over real transports through the governed path with every write attributed, but reached only a median 76 % of that optimum—a measured division of labour, reported as such. All results come from one executable pipeline with machine-checkable reproduction against frozen baselines.

Index Terms—Agent integration framework, closed-loop control, digital twin, human-in-the-loop, industrial automation, large language models, multi-agent systems, protocol heterogeneity, node-native integration.

I. INTRODUCTION

INDUSTRIAL enterprises are deploying large language model (LLM) agents widely, with surveys reporting broad adoption [1], [2]. Yet essentially all published industrial LLM systems stop at the *advisory boundary*. Multi-agent frameworks construct industrial knowledge graphs for fault diagnosis [3], generate and verify PLC code [4], [5], and parameterize simulations inside digital twins [6], [7]; beyond single-stack control interfaces [8] and simulated fault-tolerant control [9], in none does an LLM agent team’s decision become a governed write on a protocol-heterogeneous plant.

These systems share an *integration posture*: the agent is a peripheral that advises a human or a compiler, because no published framework, to our knowledge, integrates an agent team into the plant’s own protocol, data, and semantic fabric. The boundary is well motivated—an unguarded write path from a stochastic language model to physical actuators is unacceptable in production—but its consequence is that the central engineering question of agentic industrial automation has never been answered quantitatively: *what does it cost, in integration effort, latency, and human attention, to let an LLM agent team close the control loop on a real plant, and what safety does the resulting governance buy?* This paper measures the first two costs and releases the human-attention arm as a pre-registered protocol design.

This paper argues that the answer requires a framework providing three things no existing work provides together: (i) a *node-native integration substrate* that embeds agent teams into the plant’s supervisory fabric—agents, protocol drivers, and plant data share one process and one data domain, so that binding an agent to an actuator is a configuration act, not an integration project; (ii) a *governed write path* that confines the accepted-action language of every agent-issued write to what is safe by construction, attributable, and recoverable, formalized precisely enough to be checked rather than asserted; and (iii) a *reproducible evaluation protocol* that measures both the adaptability of the integration and the safety-overhead trade-off of the write path across protocols, plant scenarios, and fault conditions.

We present AGENTWORKSHOP, a configuration-driven *integration framework* that meets all three requirements, together with its evaluation suite, AW-INDUSTRIALBENCH. It couples two runtimes that are normally separate products into one process and one data domain: an *agent half* (14 interchangeable LLM harnesses, lead-based team supervision, episodic-semantic memory with hybrid retrieval, cross-channel coordination) and an *industrial half* (five protocol drivers—Modbus TCP, Modbus RTU, OPC UA, MQTT, HTTP/REST—per-node edge acquisition runtimes, a hysteresis-and-debounce alarm chain, and time-series/frame storage). The two halves meet at a *governed bridge*: nodes are bound to agents (individually or as a team), physical semantics are injected as semantic cards (unit, hard safety range, soft recipe window, warn band, freshness) so that agents never see register addresses, and the agent tool surface exposes exactly one write path—the

Manuscript received XXXX; revised XXXX. (Corresponding author: First Author.)

The authors are with XXXX, City, Country (e-mail: first.author@xxxx.edu).

governed write-control pipeline, composing a batch-scoped recipe-window *interlock*, a mode-bound human-approval *gate*, a readback-verified *write*, and a signed, attributable *journal*. A system-level *backstop* continuously scores open optimization records against the recipe window and executes bounded, non-destructive rollbacks—at most two consecutive automatic recoveries before mandatory human escalation. Because the agent runtime and the node drivers share one process, this contract is *enforceable* rather than merely declared. Commissioning is configuration, not code: driver configurations are derived at runtime from each plant device’s integration export, agent–node bindings and approval modes are declared per node, and a named *plant scenario*—devices, signal semantics, dynamics, recipes—re-instantiates the full deployment; Sec. VI-B measures this claim directly by recommissioning the same live instance onto a second, independently defined plant scenario by switching one scenario identifier.

Our contributions are:

- 1) **A node-native integration framework** for LLM multi-agent teams on heterogeneous production lines: agent runtime and industrial runtime in one process and one data domain; five protocol drivers with per-node edge acquisition; physical-semantic grounding via semantic cards; and team-level node binding under a 14-engine harness registry, so that agent intelligence remains a replaceable component (Sec. III).
- 2) **A governed write-control pipeline** for agent-issued actuator writes, formalized as a supervised state machine with three safety invariants (containment, readback closure, bounded autonomy), each mapped to implementation anchors and negative acceptance tests (Sec. IV).
- 3) **AW-INDUSTRIALBENCH**, a reproducible evaluation protocol with a quantified safety–overhead design: to our knowledge the first experimental design that measures interception rate, false-block rate, and write-latency distribution of an approval-gated agent write path, with the human-attention dimension fixed by protocol as E8 (a human-operator approval baseline), against governance ablations, general-purpose agent frameworks, human operators, classical control, and cross-engine replications. The measured arm is reported here (Secs. VI-A–VI-H); the remaining arms are released as executable protocol designs. Two quantities are measured rather than asserted: bypassing the soft interlock drops interception from 100 % to 66.7 %—breach writes execute and are journaled—while no ablation arm adds latency resolvable at this tier, so the cost of governance is structural rather than temporal.
- 4) **A completed closed-loop program on a seeded extrusion cast-film line**: a governed controller recovers $J/J^* = 97.0\text{--}97.2\%$ of the offline grid optimum ($W^*=89.894$) in two iterations and at most two governed writes per plant seed across three seeds, and four released LLM-agent-team campaigns close the same loop over real transports with every write attributed but reach only a median 76 % of that optimum (Secs. VI-G–VI-I). We report the agent result as a measured negative,

because the gap is what motivates the propose/dispose split rather than a claim to be softened.

II. RELATED WORK

A. Advisory-Boundary Systems

Industrial LLM applications cluster into advisory families. *Diagnosis and assistance* couples LLMs with knowledge structures: CoMA-IKG automates the construction of industrial knowledge graphs with an LLM-driven multiagent framework, supporting fault diagnosis [3]; surveys chart the breadth [1]. *Engineering automation* generates control code: Agents4PLC verifies generated PLC code against formal specifications [4], and MetaIndux-PLC steers generation with control-logic guidance [5]. *DT-coupled agents* parameterize simulations [6], use a twin as prompt knowledge base and validation sandbox [7]. All share two boundaries: the output reaches a human or a compiler, never an actuator, and in the DT family the twin is the *destination* of agent action, whereas in AGENTWORKSHOP the twin is one observer of a real protocol stack and the destination is the plant. *Closed-loop LLM control*, the newest family, removes the advisory boundary itself: LLM agents have converted fault detections into constraint-aware fault-tolerant control actions on simulated plants [9], and tool-equipped LLMs have controlled real machines in natural language over OPC UA [8]. LLM agents also actuate physical equipment in adjacent domains—autonomous chemical experimentation [10] and robot control [11]—but target single-apparatus loops, not production lines. These works demonstrate the feasibility this paper builds on, yet couple one agent (or team) to one protocol stack with per-application governance; none contributes a reusable integration substrate (protocol-heterogeneous, node-native, team-level binding) or a reproducible governed-write benchmark, which is the gap AGENTWORKSHOP fills. The alarm chain of Sec. III-B follows alarm-management practice (ISA-18.2) [12].

B. Governed Execution, Benchmarks, and Frameworks

Outside industry, τ -bench [13] supplies the evaluation paradigm we adopt for approval-gate semantics. Hard action enforcement has its own line—control barrier functions [14] and shielding [15]; here the wrapped component is an *agent tool surface*, and the fallback path (interlock, approval, readback, journal) is itself part of the audited contract. Approval semantics rest on levels-of-automation research [16]. The line/product/recipe/run tree of Sec. III-F combines ISA-95 level integration [17] with the ISA-88 recipe/batch model [18], and the semantic card is the node-level counterpart of the Asset Administration Shell (IEC 63278) [19]. General agent benchmarks [20] involve no physical plant or actuator safety contract; industrial datasets (TEP [21], SWaT [22]) target fault diagnosis, not agent closed-loop control—AW-INDUSTRIALBENCH bridges the two. Orchestration frameworks (AutoGen [23], LangGraph, CrewAI; on ReAct [24]) provide scheduling but no protocol layer, semantic grounding, or write governance, so they are treated as baselines over the same MCP tool surface [25]. Network segmentation follows

IEC 62443 [26], complementing IEC 61511's separation of basic control from safety functions [27]; AGENTWORKSHOP applies that separation at the agent-plant layer. The defining gap AW-INDUSTRIALBENCH occupies is the combination of node-level agent integration—one runtime, one data domain, team-level node binding—with closed-loop actuation under a formalized governance pipeline, per-write attribution, and a released benchmark.

III. SYSTEM ARCHITECTURE

AGENTWORKSHOP (v0.7.39) is an integration framework, not an application: it ships the two halves that industrial agent deployments otherwise bolt together by hand as a single Node.js process—a REST and WebSocket API, an MCP server with 25 tools [25], an A2A agent interface, and a plugin host with hot-reload and error isolation—all under one authentication layer. Operators enter through the web console or a terminal (TUI) workbench sharing the same approval queue. Fig. 1 shows the complete architecture.

A. Design Principles

P1, node-native integration: the agent runtime and the industrial runtime execute in one process and one data domain—binding, telemetry, and write control share invariants rather than APIs. This is what makes the write-path contract of Sec. IV *enforceable*: gateway-style integrations must trust a translation layer they do not control, whereas here the governance checks run in the same process as the driver calls they guard. *P2, supervisory positioning:* AGENTWORKSHOP occupies the supervisory, soft-real-time tier; hard real-time control and safety functions remain in PLCs, mirroring the IEC 61511 separation of basic control from safety instrumented functions [27]. *P3, configuration-driven, down to the plant scenario:* a typed configuration schema drives acquisition intervals, approval policies, retention, and memory budgets; driver configurations are derived from each device's integration export rather than hand-written, and a named *plant scenario*—devices, signal semantics, dynamics, recipes—re-instantiates the full deployment, so adapting to a new line is a configuration task (measured in Sec. VI-B: the same instance is recommissioned onto a second scenario by switching one identifier, with zero platform code changes). *P4, engine-agnostic governance:* agent intelligence is a replaceable component; the governance pipeline of Sec. IV is defined at the tool boundary and is therefore identical for every engine.

B. Industrial Half

Five protocol drivers spanning four protocol families—Modbus TCP, Modbus RTU (over TCP), OPC UA [28], MQTT, and HTTP/REST—sit behind a uniform driver interface and are plugin-registerable at runtime (`ctx.daq.registerDriver`); a mock driver supports deterministic testing. Each acquisition node runs an *edge runtime* that owns its sampling schedule and in-flight mutual exclusion, organized around three decoupled intervals per node: acquisition (default 5 s, floor 1 s, hard clamp [0.1, 60] s; sub-tick

intervals are served on the next 250 ms sweep tick), Web-Socket publish (default 1 s), and display/trend (default 5 s)—decoupling acquisition from delivery prevents downstream consumers from starving the plant-facing schedule. Nodes are sampled by a gateway sweep with a 250 ms tick and a non-accumulating budget of 64 in-flight samples per tick; a *rotating cursor* plus a per-node *due check* prevents budget from being consumed by not-yet-due nodes ahead of due ones, so tail nodes are never starved.

Each node also runs an alarm state machine over `ok/warn/alarm/offline`: an 8 % warn band, 2 % hysteresis, three-frame debounce on `ok↔warn`, watermark-based out-of-order defense, and recipe-window violations overlaid as edge-triggered `alarm`; acknowledgment is a human step with escalation (default 15 min). Time series persist to TimescaleDB (batched 500 ms flushes, retention 168 h/720 h) and frames to MinIO with degraded fallback to local disk; platform state persists to SQLite in WAL mode with an FTS5 index, and every alarm, write, recipe change, and rollback row carries line, product, and recipe dimensions, enabling the metric slices of Sec. VI.

C. Agent Half

Fourteen engines are registered with a capability matrix (steer, supervise, hitl, terminal, context statistics, compaction), spanning an in-process deterministic mock (used for the experiments of Sec. VI), a first-party RPC engine (`omp`), and vendor or open-source CLI/SDK agents (`codex`, `claude`, `opencode`, `qwen`, `gemini`, `pi`, `hermes`, `dsh`); capabilities are asserted per engine, verified by the static audit (Sec. VI-C), and dispatch happens only after a three-stage availability probe. Work is organized as a seven-state task tree: each team's *lead* runs a supervision loop (1 s tick with idle backoff; event wake-up) whose LLM decisions are throttled by a state fingerprint—an unchanged world state skips the LLM call—and always shadowed by a deterministic rule engine (FIFO dispatch, shortest-queue-then-longest-idle selection, retry limit 3, 300 s stall watchdog with tool-activity liveness) that guarantees progress if the LLM abstains, errors, or produces nothing actionable; both mechanisms are ablated as E2/E3 in the released protocol. Agents accumulate typed memories (session, task, peer, team chronicle, reflection, semantic) in SQLite with FTS5 and an optional vector index, retrieved by reciprocal-rank fusion [29] ($k=60$), scored by $0.5\rho + 0.3\phi + 0.2\iota$ (relevance, recency, importance), diversified by maximal-marginal-relevance [30] ($\lambda=0.7$), and injected in three layers under an explicit 500-token budget; compaction triggers at 70 % of the context window.

D. The Governed Bridge

The bridge is where the integration claim becomes concrete: a commissioning engineer binds plant nodes to agents (a single agent or a whole team), the platform injects physical semantics into the bound agents' context, and the bound agents' tool surface is derived from the binding—there is no separate integration layer to build or maintain. A bound node yields a *semantic card* in the agent's context: physical quantity,

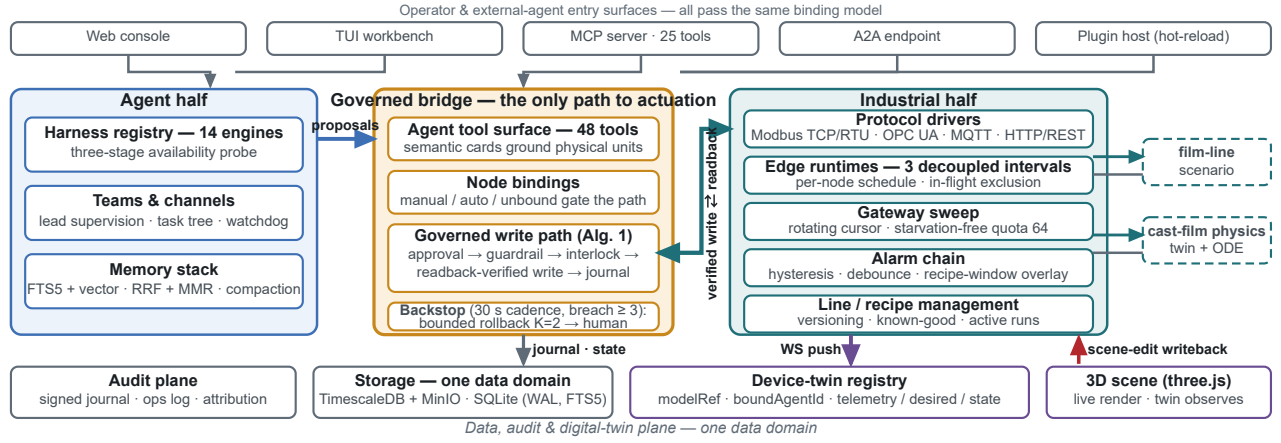


Fig. 1. AGENTWORKSHOP integration framework. **Left (agent half):** 14-engine harness registry, lead-supervised teams, episodic–semantic memory. **Center (governed bridge, amber):** the only path from agent intent to actuation—semantic cards, node bindings (manual/auto/unbound), the governed write path of Algorithm 1, bounded-autonomy backstop. **Right (industrial half):** five protocol drivers, edge runtimes, starvation-free gateway sweep, alarm chain, line/recipe management; connectivity ends at real devices or the two seeded plant scenarios. **Bottom (one data domain):** audit journal, storage, device-twin registry feeding the three.js scene over the WebSocket bus (purple); the twin renders live state but never actuates (red).

unit and display precision, the *hard interlock* range, the *soft interlock* recipe window, warn band, live value with freshness, and an operating rule limiting any single step to 2% of span. Bindings carry an operating mode: *manual* (every agent write is approval-gated), *auto* (writes proceed under the interlock and backstop), or *unbound* (no write path); a team’s members can share one node set, so a lead-dispatched task and a worker’s actuation share one authorization scope. Agents never see register addresses.

Agents act through a host-tool surface of 48 tools, 23 of them industrial or automated-ML (the MCP server exposes a 25-tool subset): setpoint control (`dcw_control`), reading, frame retrieval, judgment, rollback execution, audit views, and recipe editing under approval. Two separations are enforced architecturally: *judgment is not execution*—`dcw_judge` records a keep/rollback/uncertain verdict but cannot actuate—and *lead-only tools* are restricted to lead agents with approval. Sec. IV formalizes the single write path.

E. Digital Twin and Node-Binding Rendering

Rendering binding is part of the integration model, not a visualization afterthought. A *device-twin registry* persists, per scene entity, the bound 3D asset (`modelRef`), the agent in charge (`boundAgentId`—a virtual operator avatar acting through MCP `device.* tools`), an entity kind (device, environment, asset, or procedural sensor mesh), and the three state planes of the twin contract (the device-shadow pattern [31]): *telemetry* (live fields from acquisition or simulation), *desired* (the agent’s intent), and *state* (the reconciled value), plus the command set. A WebSocket event bus broadcasts device lifecycle events and every acquisition reading, so the scene re-renders live values without polling; conversely, scene edits write transforms back to the node registry, so the scene doubles as a commissioning editor while never becoming a source of actuation: *desired-to-state* reconciliation itself traverses the governed write path when it actuates (Sec. IV).

F. Production-Line Management

The framework models a production line as a four-level object tree—line, product, recipe, *run*—because every governance object of Sec. IV hangs off it: the recipe window constrains writes jointly with the hard range while a run is active (Eq. (1)), acquisition samples only nodes whose line has an active batch (between batches, PLC-side protection persists per P2), and the backstop scores open records against the run’s monitoring window. The recipe lifecycle is versioned and non-destructive: editing creates version $n+1$; marking the active run *known-good* freezes the rollback baseline; reverting to an earlier version creates a new version rather than rewriting history; *known-good rollback* re-dispatches the frozen parameter set to the PLC; and one-click *apply* opens a fresh batch. An active run therefore binds exactly one recipe version to one batch, giving every write a unique governance context—the window $W_r(p)$ of Eq. (1)—and giving every alarm, journal anchor, and backstop evaluation an unambiguous owner. The lifecycle has two bounded exits from an off-window excursion: automatic rollback while the per-node chain counter is below $K=2$, and mandatory human escalation afterwards. Every operation lands in a central audit log with typed actor attribution (user, agent via channel/member badge, system) and line/product/recipe dimensions; write operations are additionally anchored in a signed journal with previous/new values, source, approval, and record identifiers—the substrate for the per-write attribution metric of AW-INDUSTRIALBENCH (Sec. V) and the forensic queries used in the case study (Sec. VI-I).

IV. THE GOVERNED WRITE-CONTROL PIPELINE

This section formalizes the single path by which an agent actuator request becomes a plant write. Throughout, a *device* denotes one integrated piece of plant equipment and a *node* n one write-able parameter of such a device (a device may expose several nodes); n carries a hard safety range $S(n) = [n^{\min}, n^{\max}]$ and a value transform t_n (linear scale/offset with nonzero scale, or discrete mapping). An active *recipe run* r

over the node's line prescribes a process window $W_r(p) = [p^{\min}, p^{\max}]$ for parameter p of n .

A. Effective Constraint

Define the *effective constraint*

$$C(n, r) = \begin{cases} S(n) \cap W_r(p), & \text{run } r \text{ active,} \\ S(n), & \text{otherwise,} \end{cases} \quad (1)$$

During an active batch the accepted set is the *intersection* of the hard range and the recipe window, enforced by two independent mechanisms: a value inside $S(n)$ but outside $W_r(p)$ is rejected by the interlock (its error message cites the window as the binding constraint), while a value outside $S(n)$ is rejected by the driver's engineering-quantity validation, which applies in every state. Containment therefore does not depend on the design-time property $W_r(p) \subseteq S(n)$; authoring-time containment validation is a hardening item (Sec. VII-A) that the intersection semantics absorbs at runtime. Both runtime branches of Eq. (1) are exercised dynamically in Sec. VI-D.

B. Pipeline Stages

Algorithm 1 specifies the pipeline. A write request $q = (n, v, src, a)$ from source $src \in \{\text{agent, manual, recipe, rollback}\}$ and actor a passes an *approval stage* at the tool boundary, then six ordered execution stages enforced by the single write entry—approval is a *decision*; execution is a *validated act*.

Stage 0 (approval gate; manual bindings only). For agent proposals on manually bound nodes, the tool layer routes the proposal to an approval request (deduplicated per node and record) and does not submit it further until a human decides; unapproved requests lapse with their record (30-min stale takeover), defaulting to non-execution. A binding is a persistent (agent, node) association whose mode—*manual, auto, unbound*—selects the backstop's rollback policy (approve_rollback / auto_rollback / observe_only). Approval is decoupled from execution: every stage below re-runs after the decision, so a recipe change during deliberation cannot be exploited through a stale approval. Auto-mode bindings skip Stage 0 but remain subject to the backstop (Sec. IV-C); unbound bindings cannot write at all—the gate is a property of the binding model, not a prompt. Decisions carry actor identity and latency (Sec. VI-F); the human-operator study E8 remains specified, and agents keep observing telemetry while pending.

Stage 1 (availability gate). If the node's gateway is paused or the node is disabled, the request is rejected—governance is unavailable exactly when actuation is unavailable; no bypass exists.

Stage 2 (closed-loop guardrail). Agent-sourced writes are serialized per optimization scope—one open record per agent; records idle beyond $T_{\text{stale}} = 30 \text{ min}$ are superseded by the next writer. Within a rollback cooldown $T_{\text{cd}} = 300 \text{ s}$, a same-direction rewrite is rejected while a reverse-direction correction remains allowed, preventing oscillatory overwrites.

Stage 3 (soft interlock). The proposed value must satisfy $v \in C(n, r)$ (Eq. 1); violations are rejected before any driver interaction.

Algorithm 1 Governed write path $\text{WRITE}(n, v, src, a)$

Require: node n , value v , source src , actor a ; active run r if any

- 1: **if** $src = \text{agent} \wedge \text{binding}(n) = \text{manual}$ **then**
- 2: route to approval; **return** pending $\triangleright$ Stage 0: resumes at line 4 post-approval; lapse $\Rightarrow$ never executed
- 3: **end if**
- 4: **if** $\text{binding}(n) = \text{unbound}$ **then**
- 5: **return** reject(FORBIDDEN) $\triangleright$ tool surface never dispatches it
- 6: **end if**
- 7: **if** gateway paused $\vee \neg n.\text{enabled}$ **then**
- 8: **return** reject(CONFLICT)
- 9: **end if**
- 10: **if** $src = \text{agent}$ **then**
- 11: **if** $\exists$ open record r' of $a \wedge \text{age}(r') < T_{\text{stale}}$ **then**
- 12: **return** reject(CONFLICT)
- 13: **else if** $\exists$ open record r' **then**
- 14: SUPERSEDE(r')
- 15: **end if**
- 16: **if** same-direction write within T_{cd} **then**
- 17: **return** reject(CONFLICT)
- 18: **end if**
- 19: **end if**
- 20: $W \leftarrow C(n, r)$ $\triangleright$ Eq. (1)
- 21: **if** $v \notin W$ **then**
- 22: **return** reject(soft interlock)
- 23: **end if**
- 24: $(raw, rb) \leftarrow \text{driver.write}(n, t_n^{-1}(v))$
- 25: $v_{\text{rb}} \leftarrow t_n(rb)$
- 26: $ok \leftarrow |v_{\text{rb}} - v| \leq \tau(n)$ $\triangleright$ Eq. (2)
- 27: JOURNALAPPEND($n, v^{\text{prev}}, v, src, a, r$)
- 28: AUDIT(write, n, v, a) $\triangleright$ 10 s dedup
- 29: **return** (ok, v_{rb})

Stage 4 (write with readback). The value is encoded through t_n , sent by the driver, and read back; the readback is decoded and compared under tolerance

$$\tau(n) = \frac{1}{|t_n^{\text{scale}}|} \max(0.5 \cdot 10^{-d(n)}, 0.005(n^{\max} - n^{\min})), \quad (2)$$

where $d(n)$ is the display precision: the larger of half a least-count digit and 0.5 % of span, scaled by calibration. A write is *ok* only if the decoded readback agrees within $\tau(n)$; if the driver cannot read back, the result is flagged degraded rather than silently confirmed.

Stage 5 (journal anchor). Every executed write appends a signed journal anchor ($id, line, n, prev, new, src, a, r, approval, record, t$).

Stage 6 (audit). The audit plane receives a typed write event attributed to the actor's kind (agent writes carry the channel/member badge); identical node-value events within 10 s are deduplicated.

C. System Backstop and Bounded Autonomy

An optimization record opened by `dcw_control` is re-evaluated every $T_{\text{re}} = 30 \text{ s}$ once it is older than $T_{\text{win}} = 120 \text{ s}$,

against a baseline window $T_{bl} = 600$ s and the record parameter's active recipe window $W_r(p)$. With b the number of out-of-window samples: if $b \geq B = 3$ and the binding policy is *auto*, the system issues a rollback verdict and executes it only while the record's automatic-rollback chain is below $K = 2$; otherwise the verdict is recorded and escalated to a human. Rollback is *non-destructive* at four granularities (record pre-set baseline, per-node last stable anchor, batch-start values, frozen known-good snapshot) and every rollback write traverses the same governed pipeline; a paused gateway defers recovery with actuation (fail-safe direction). Chain boundary I3 is per record chain, so a rolling per-node budget of automatic writes is stated as a limit (Sec. VII-A).

D. Trust Boundary and Threats

The pipeline guarantees the *sanctioned* path; no framework can constrain software that bypasses it. Because harness engines are privileged local processes—a compromised engine could write to the protocol stacks directly, the failure mode indirect prompt injection targets [32]—the deployment model uses a three-stage availability (allow-list) probe (Sec. III-C), a least-privilege service account, and IEC 62443 network segmentation [26] to keep the governed tool surface the only *reachable* write path, trading 62443 zoning isolation for in-process enforceability within one zone. Inside the sanctioned path a prompt cannot *enlarge* an agent's authority; it can only abuse an authority the operator already granted, which is what the approval gate and the chain limit bound. Governance is fail-safe under storage degradation (F6): acquisition continues and the audit plane reconciles on recovery.

E. Safety Invariants

The pipeline is designed to maintain three invariants, each mapped to an implementation anchor and to the platform's scripted acceptance suite, a dedicated subset of which exercises rejection paths:

- **I1 (containment).** Every executed agent write satisfies $v \in C(n, r)$ (Eq. (1), both branches). *Anchor:* single-entry write function. *Evidence:* rejection-path acceptance tests; live-instance verification (Sec. VI-D).
- **I2 (readback closure).** A write is reported *ok* only if the decoded readback lies within $\tau(n)$ or the result carries an explicit degraded flag. *Anchor:* Eq. (2) in the write executor. *Evidence:* deadband and calibration tests; the no-readback ablation.
- **I3 (bounded autonomy).** Unattended corrective writes per record chain are at most $K = 2$; the third response and all manual-policy rollbacks require human confirmation; stale records are superseded, never accumulated. *Anchor:* chain counter with escalation branch. *Evidence:* rollback-manager acceptance tests; the backstop drill (Sec. VI-F).

These invariants are stated over the pipeline, not over the LLM: whatever the agent proposes, the accepted-action language is constrained. This is the precise sense in which the system's autonomy is *governed*.

TABLE I
DATA-CREDIBILITY LAYERS; CLAIMS ARE LABELED BY THE HIGHEST LAYER MEASURED, AND EVIDENCE IS NEVER POOLED.

Layer	Description	Status
L1	Seeded simulation: two independently defined plant scenarios (strategy-signal film line; ODE cast-film twin), seeded noise, offline optimum W^*	measured
L2	TEP [21] / SWaT [22] replay through real drivers	designed
L3	One physical OPC UA/Modbus device on the real stack; write-readback latency	designed
L4	Digital-twin case study: seeded closed-loop campaign on the cast-film line—deterministic controller over three seeds and an agent team over four runs (Secs. VI-G, VI-I)	measured

V. AW-INDUSTRIALBENCH: BENCHMARK AND EXPERIMENTAL PROTOCOL

A. Design Principles and Data Layers

AW-INDUSTRIALBENCH is built on three principles. *Determinism:* a seeded PRNG (mulberry32) and a scenario-snapshot API make structural metrics (interception verdicts, judge-class checks, the deterministic setpoint path) point-wise reproducible; objective values computed from *sampled* process variables remain timing-sensitive (Sec. VI-J). An offline grid search exposes the optimum W^* as the reference bound. *Layered realism:* results are reported per data layer (Table I), never pooled, so claims carry their realism level explicitly. *Executable protocol:* the released audit harness (bench/run.mjs; four tiers, seeded, results never overwritten) executes the artifact and invariant checks of Secs. VI-C–VI-D today; the campaign configurations for the E1–E9 families are declarative documents (E-numbers are campaign instantiations of the B groups; E8 = B4), and the campaign runner, metrics extractor, and statistics module that consume them are specified in the released plan (Sec. VI-A states precisely which parts are running and which are in development).

B. Tasks, Faults, Metrics, and Baselines

Five task families cover the supervisory workload, each with a machine-readable oracle evaluated against the database, readback values, and event streams—never against log strings: *T1* closed-loop setpoint correction in-window; *T2* out-of-window excursion; *T3* multi-node joint adjustment under the step rule; *T4* batch parameter recovery; *T5* mid-run objective change and re-planning. Six fault families are defined for injection at the simulator control API: F1 step drift, F2 sensor freeze, F3 communication interruption (30 s), F4 out-of-order injection, F5 out-of-window writes (adversarial proposals), F6 storage unavailability; F5 stresses governance (I1), F6 degraded-mode integrity.

Metrics form three families. *Safety* metrics require the governed pipeline: out-of-window write interception rate (denominator = seeded attacks), readback-failure capture rate, false-block rate (denominator = legal in-window writes attempted), MTTR after fault, spurious-rollback rate, HITL-blocked harmful-write share, and per-write attribution rate (target 100 %). *Performance* metrics cover the *automatic segment*

(proposal acceptance to verified write), kept separate from HITL waiting time so human response time is never conflated with system latency. *Cost/quality* metrics account task goal rate, tokens and LLM calls per completed task, approval wait time, human interventions per task, and long-horizon token area-under-curve.

Eight baseline groups are defined. B1 ablates the pipeline’s own mechanisms (bench-mode-only bypass switches). B2–B3 isolate the platform from its intelligence: general frameworks (AutoGen [23], LangGraph, CrewAI; ReAct) on the same MCP tool surface [25]. B4 is a human-operator reference under the τ -bench paradigm [13]; B5 anchors against classical control with W^* as reference; B6–B7 are the scheduling and memory arms of Sec. III-C; B8 replicates tasks across real engines.

Reproducibility protocol. Two tiers are distinguished honestly. *Released and running today:* the harness executes every check of Secs. VI-C–VI-D from a single entry point with fixed seeds; every run records its seed, harness source hash, and commit, appends to never-overwritten timestamped directories, and produces judge-class metrics identical across fixed-seed executions (latency varies with the environment and is reported as ranges). *Specified, in development:* the campaign runner, the statistics module (Mann–Whitney U with Holm correction, bootstrap 95 % CIs; unit of analysis = run), the fault injector, and the decision-log instrumentation persisting each LLM decision with its prompt hash, model version, and token usage. Real-LLM arms run temperature-0 with pinned model versions. The platform is released under PolyForm Noncommercial: academic reproduction is permitted; commercial reuse is a licensing boundary, not a reproducibility one.

VI. EXPERIMENTAL EVALUATION

A. Setup

All results in this section were produced by one executable pipeline (`bench/pipeline.mjs`) that boots the plant simulator and the platform, provisions the plant over real protocol transports, drives the governance surface, runs the closed-loop campaign, and writes a machine-readable archive. A single command reproduces the section:

```
node bench/pipeline.mjs --profile integrated --seed
42 --cl-seeds 3
```

The pipeline emits no result it did not measure: every reported number below is read from the evidence records of the never-overwritten run directory. Each run records its seed, the harness source hash (SHA-256 over the eight pipeline checker sources, 204fc9ee here), the repository commit (d6c824d), the Node/platform, and the full evidence chain. Host environment: an AMD Ryzen 7 9700X (8 cores) host with 31 GB RAM running Windows.

The pipeline executes fourteen stages: preflight and bootstrap (P0), plant-model instantiation with the offline optimum (P1), multi-line provisioning over five protocol drivers (P2), acquisition, write-control and interlock checks (P3), the deterministic tool-level control loop over three observe–actuate–judge cycles per line (P4) with rollback/record/ledger surface (P4b), approval gate (P4c), audit reads (P4d), recipe lifecycle

(P4e), the closed-loop optimization benchmark (P6), multi-modal acquisition (P7), the cross-scenario portability check (P8, Sec. VI-B), the system-backstop drill (P8b, Sec. VI-F), and the platform-subsystem benchmarks (P9, Sec. VI-F). Stage numbering follows the released protocol (no P5). A full run over five devices completes in about 5.2 min; the closed-loop benchmark contributes about 36 s per plant seed and the backstop drill a cadence-bound ~ 120 s observation window.

Headline results. Across both scenarios the pipeline intercepted 33/33 seeded out-of-window/range writes (Wilson 95 % [89.6 %, 100 %], zero false blocks), recommissioned the deployment onto the second scenario with zero platform code changes, recovered 97.0–97.2 % of the offline grid optimum in the governed closed loop under a fixed controller, and exercised the backstop end-to-end (~ 120 s). An LLM agent team closed the same loop over real transports through the same path, but reached only a median 0.76 of that optimum (Sec. VI-I).

Scope. The reported run exercises tasks T1–T2 and faults F2/F3/F5 against baseline group B1; T3–T5, F1/F4/F6, hardware-in-the-loop, public-dataset replay, cross-engine replication, and baselines B2–B8 are covered by the released campaign designs and are *not* reported here. All measured layers are simulation-tier (Table I).

B. Cross-Scenario Portability (Framework Claim)

The framework’s central claim—adapting the deployment to a new production line is a configuration task, not an integration project—is measured directly by the pipeline’s final stage. The pipeline first provisions everything below on the *cast-film physics* scenario (ODE plant model, six actuators, seven sensors, five protocol drivers, vector and image frames). It then switches the plant to a second, independently defined scenario, *film-line*: a coating-oven line with a disjoint signal domain, driven by per-signal strategies rather than a physics engine. No scenario-specific code exists in the commissioning path: the identical five-step flow (export $\rightarrow$ connection test $\rightarrow$ node creation $\rightarrow$ line/product/recipe provisioning $\rightarrow$ agent binding) re-instantiates the deployment on both scenarios, because driver configurations are derived from each device’s export at runtime (Sec. III-A, P3).

Measured on the second scenario: five devices across all five protocol drivers are commissioned in one pass—the commissioning artifact is five driver configurations derived from the devices’ exports plus one scenario identifier—three writable-setpoint devices become governed lines (Modbus TCP, OPC UA, MQTT), the two acquisition-only devices attach as satellites; all five nodes stream live data (9–12 one-second buckets); governed writes are accepted with readback deviation 0.04 °C on Modbus TCP and exact on OPC UA; and all nine seeded out-of-window attacks are rejected with zero false blocks (0/3 legal in-window writes). The stage completes in about half a minute and the rig is restored. The structural verdicts are identical across scenarios—100 % interception, 0 % false blocks: per-line engineering moved into authoring the scenario definition, while the platform absorbed the difference unchanged.

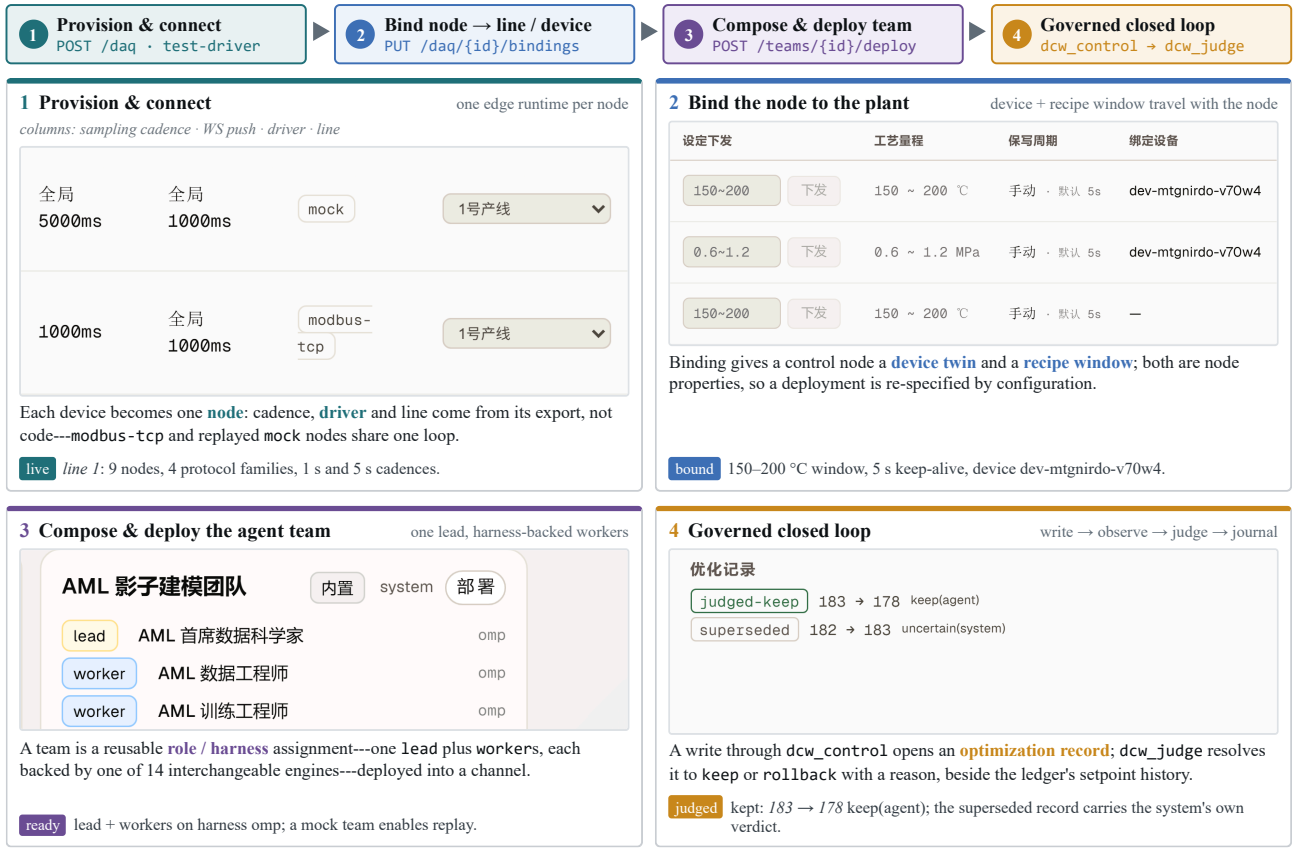


Fig. 2. Operational walkthrough of the framework, captured on the live instance (screen capture of the released artifact; seeded plants). A device becomes a *node* whose protocol driver and sampling cadence are configuration (1); the node is *bound* to a device twin and a recipe window, both node properties rather than code (2); a team of one lead and harness-backed workers is *composed* in a reusable library and deployed into a channel (3); and every subsequent agent write opens an optimization record that *dcw_judge* resolves to *keep* or *rollback* with a reasoned verdict, shown here against the setpoint ledger (4). The ribbon names the entry point of each stage. All four surfaces exist in the shipped platform, and none contains scenario-specific code.

Fig. 2 shows that same commissioning path as an operator drives it on the live instance, and the four surfaces that carry it. (1) *Provisioning*: a device is registered as a *node* whose protocol driver, sampling cadence and line are configuration derived from the device's integration export, so a real Modbus-TCP node and replayed mock nodes coexist in one acquisition loop, and a connection test gates creation. (2) *Binding*: a control node is attached to a device twin and a recipe window, which is why the process window travels with the deployment rather than with a line-specific program. (3) *Team composition*: the agent side is a *team*—exactly one lead plus harness-backed workers—deployed as a unit into a channel; because the harness is a replaceable component, one team definition runs on an LLM engine or on the deterministic replay engine. (4) *Operation*: from then on the plant speaks back through the governed loop—each write opens an optimization record, *dcw_judge* resolves it to *keep* or *rollback* with a reason, and the ledger renders the record history against the setpoint history annotated by source (*agent*, *manual*, *recipe*, *rollback*). Together these four surfaces are the whole commissioning-and-operation path, and no plant-specific code appears in any of them: this is the integration claim of Sec. III made operational.

C. Static Capability and Artifact-Consistency Audit

Before any dynamic campaign, the released audit harness (*bench/run.mjs*) verifies the artifact at the source level, because a paper whose constants cannot be traced to code is not reproducible in any meaningful sense. Three checks run offline: a *capability inventory* against the claims of Sec. III; a *paper-code consistency* check that locates 20 representative quantitative constants cited in this paper (sweep quota 64, breach threshold 3, rollback-chain limit 2, RRF $K=60$, MMR $\lambda=0.7$, compaction 0.7, watchdog 300 s, hysteresis 2 %, and others) at their implementation anchors; and a *pipeline stage-ordering* check asserting that the governed-write stages of Algorithm 1 appear in the declared order at their source anchors. All pass: 20/20 anchors and 2/2 ordering chains—engineering discipline rather than validation evidence, since the harness audits the artifact it belongs to; a future edit that invalidates a claim fails this audit rather than surfacing in review.

D. Dynamic Invariant Verification on a Live Instance (L1-Tier API Surface)

The static audit verifies structure; this subsection verifies *behavior*: L1-tier evidence (Table I), driving a pinned platform

instance (v0.7.39, mock drivers) exclusively through its REST API with seeded randomness (mulberry32, seed 42). A benchmark line is created with global safety range $[120, 260]^\circ\text{C}$ and a batch is started with active recipe window $[175, 205]^\circ\text{C}$; the safety invariants of Sec. IV-E are then attacked and probed, including the decision boundaries and both branches of Eq. (1).

I1 (containment). Six seeded gross out-of-constraint writes—global-range violations (107, 269), window violations while the batch is active (219, 163), and extreme/negative values (10^6 , -25)—are all rejected with typed validation errors; simultaneously, all three in-window control writes are accepted, so the interception rate is 100% at a false-block rate of 0%. Boundary probes at $\pm$ one least-count behave as specified (205.0 accepted; 205.1, 174.9 rejected). The rejection message cites the window as the binding constraint—the global range is stated not to apply to the running batch (original retained in the released evidence). With the batch stopped, the window branch deactivates and the hard range alone governs: an out-of-range write (300) remains rejected while a within-range, out-of-window write (130) is accepted—confirming that the soft interlock is a batch-scoped constraint, not a constant.

I2 (readback closure) and attribution. All accepted writes return a driver readback within the tolerance of Eq. (2) ($\tau = 0.7^\circ\text{C}$ at one decimal digit and a 140°C span), and all produce journal anchors with source, actor, timestamp, and previous/new values—a 100% attribution rate. Acquisition held the 1 s cadence, and the samples API answered 20 probes at p50 6–22 ms / p95 8–63 ms.

Reproducibility. Every execution on a pinned harness—the two identical-harness pipeline runs of Sec. VI-J and the released ablation runs (three aggregated in Sec. VI-H)—produced identical judge-class metrics: the same seeded attacks, rejections, boundary verdicts, and ledger outcomes; only fixture identifiers and latency varied. A machine verdict is part of the archive: `bench/compare.mjs` re-runs the pipeline and asserts judge-class equality against the archived baseline (REPRODUCIBLE; the gate rejects a tampered metric, validated by a self-test). The Wilson 95% interval for 6/6 trials ([61%, 100%]) is the honest reading of any small- n rate.

Scope. These checks exercise the pipeline’s accepted-action language at the API boundary with the LLM out of the loop—deliberately so, since the invariants are defined over the pipeline, not over any engine (Sec. III-A, P4).

E. Real-Protocol Plant Scenarios (Five-Protocol PLC Node Simulator)

The API-tier checks of Secs. VI-D and VI-H use in-process mock drivers; this subsection re-runs the working-condition scenarios over *real protocol stacks*, against an independent five-protocol PLC node simulator (Modbus TCP/RTU slaves, OPC UA server, MQTT broker/publisher, HTTP endpoints, with an ODE plant model). Driver configurations are derived at runtime from the simulator’s integration-export API—no hand-written connection settings—and every candidate driver passes the platform’s connection test before a node is created.

The pipeline provisions one production line per device that exposes a writable setpoint (4 lines: Modbus TCP, OPC UA,

TABLE II
REAL-PROTOCOL PLANT SCENARIOS OVER THE FIVE-PROTOCOL PLC NODE SIMULATOR (L1, SEED 42; MEASURED). FOUR PRODUCTION LINES PLUS ONE ACQUISITION-ONLY SATELLITE NODE. WRITE LATENCY: P50/P95 OVER SIX IN-WINDOW GOVERNED WRITES PER LINE. “N/A”: PUBLISH-ONLY (MQTT) OR ECHO-ONLY (HTTP) ACTUATORS REPORT NO READBACK.

Protocol	Acq. pts	Write p50/p95 (ms)	Readback	F5
Modbus TCP	2	111.5 / 141.6	0.04 °C	6/6
OPC UA	11	19.0 / 22.1	exact	6/6
MQTT	12	19.4 / 19.5	n/a	6/6
HTTP	12	32.8 / 33.2	n/a	6/6
Modbus RTU [†]	14	—	—	—

[†]acquisition-only satellite node (gel counter).

MQTT, HTTP) and attaches the acquisition-only device (a Modbus RTU gel counter) as a *satellite* acquisition node on the first line, because the platform’s acquisition gate samples only nodes whose line has an active batch. All five nodes stream live data through their real transports (2–14 one-second buckets in the settle window, Table II). A governed setpoint write costs 19–112 ms (p50) depending on the protocol, with the Modbus TCP transaction dominating and its p95 reflecting the first write’s connection setup on the freshly created fixture; readback closes within 0.04 °C on Modbus TCP and exactly on OPC UA, while the MQTT and HTTP control channels report no readback by design (publish-only / echo-only actuators)—a driver property the write path records rather than conceals. Six seeded out-of-window and out-of-range attacks are rejected on *every* writable protocol (6/6 each; 24/24 pooled, Wilson 95% interval [86%, 100%]) with zero false blocks on the four legal in-window writes, and the deterministic tool-level control loop (`daq_query`→`dcw_control`→`dcw_judge`) converges on all four lines. A link-interruption drill (simulator protocol service stopped and restarted) is both detected by the platform and recovered, and a process-freeze injection demonstrates defense in depth: the governed write still passes (readback verifies the register, not the process), and the failure is caught within 2 s by the independent recipe-window monitor—precisely the layer separation of Secs. VI-D and IV-D.

F. Governance Surface: Recipe Lifecycle, Rollback, and Approval

This subsection measures the *lifecycle objects* the governance stack manages, because a governed loop that cannot version, revert, and attribute its own recipes is not deployable. Each probe below ran end-to-end in one pipeline run, with the LLM out of the loop. *Non-destructive history*: a revert *creates* version 3 rather than overwriting. *Judge-execute separation*: judging `rollback` changed no PLC value (196.60 °C throughout)—only the separately invoked rollback restored the origin value (203.40 °C); an agent’s verdict is an intent record, not an actuation. *Bounded autonomy in action (backstop drill)*: freezing the process variable out of window under an open auto-binding record triggers the backstop—the record is judged `rollback by the system` and the PLC restored to the baseline ($186.2 \pm 0.75^\circ\text{C}$), cadence-bound at ~ 120 s;

chain limit $K=2$ remains specified (Sec. VII-A). Default non-execution: in manual mode an agent write blocks until the approval releases it (148.2 rpm on OPC UA; decision latency 456 ms, approval-poll inclusive); the journal holds ten anchors over four sources and both log surfaces return full 200-entry windows.

Platform subsystems (P9): a mock lead dispatches a child task that a worker completes (parent COMPLETED); duplicate team-memory writes under one deduplication key yield one entry; the registry enumerates 14 engines (12 environment-available here).

G. Closed-Loop Optimization Benchmark

This subsection quantifies how much of the offline optimum a *governed* closed loop recovers on the cast-film twin. The pipeline instantiates all six actuators (zones on Modbus TCP, screw on OPC UA, line speed on MQTT, die gap on HTTP), starts a batch off-optimum ($N=150$ rpm, $v=95$ m/min, zone 200 °C), and runs a controller that reads through the acquisition path and writes through the governed path. The objective

$$J = 55 J_{\text{thick}} + 25 J_{\text{quality}} + 8 J_{\text{energy}} + 7 J_{\text{throughput}} \quad (3)$$

constrains melt temperature to $[195, 225]$ °C and pressure to ≤ 22 MPa.

The controller is model-light and fixed in advance: it inverts $h \propto N/v$ at constant line speed ($N \leftarrow N \cdot (50/h)$, ± 22 rpm per step) and corrects the heater setpoint only when melt temperature leaves $[196, 224]$ °C. It contains no LLM, so the benchmark isolates the *governed loop* rather than a particular model. Convergence requires two consecutive evaluations with $h \in [48, 52]$ μm , defect $< 2\%$, pressure ≤ 22 MPa.

Fig. 3 reports the trajectories; J_{end} is the mean of the two consecutive in-band evaluations (the plotted iteration-1/2 pair). From $J_0 = 64.7\text{--}70.2$ the loop reaches $J_{\text{end}} = 87.2\text{--}87.4$ against the offline grid optimum $W^* = 89.894$ (weights unnormalized, 0–95 scale; real-time physics), i.e. 97.0–97.2 % of the optimum, converging in two iterations with at most two governed writes per seed, no rejected write and no constraint violation; mean wall time is 36 s per seed. Thickness moves from 55.2–56.1 μm (target 50 ± 2 μm), defect settles at or below 0.64 %, pressure at 16.6–16.9 MPa, and melt temperature stays inside $[195, 225]$ °C. Per-seed values ($J_0 \rightarrow J_{\text{end}}$, ratio): seed 42: 68.07→87.26 (0.971); seed 43: 70.21→87.16 (0.970); seed 44: 64.69→87.37 (0.972). Repeating the campaign across the day’s released runs keeps $J/J^* \in [0.966, 0.973]$ (three runs $\times$ three seeds; the three run-level means are 0.971 ± 0.001 , sd, $n=3$); the residual spread reflects the sampled process variables from which J is computed. The 3 % shortfall is a throughput/energy trade, not a control failure: the grid optimum sits at a low-throughput corner where the objective’s energy weight slightly exceeds its throughput weight, while the loop holds the higher line speed it started from.

H. Ela: Pipeline-Tier Governance Ablation (Completed)

Using the env-gated bench-mode bypass (production builds refuse the flag), we compare the full pipeline

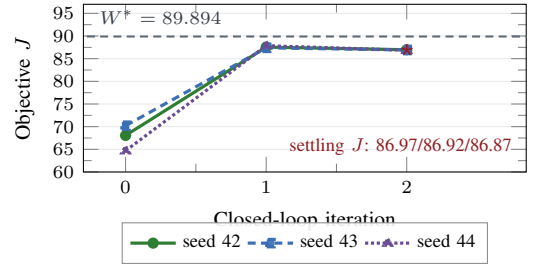


Fig. 3. Governed closed-loop optimization on the cast-film twin (three plant seeds, real protocol transports; deterministic controller, one run per seed). J is the composite objective of Eq. 3; all seeds converge in two iterations to 97.0–97.2 % of the offline grid optimum W^* (dashed). Red stars mark the iteration-2 settling measurement; the quoted ratios are the J_{end} means of both evaluations over W^* .

against the three ablation arms of B1—soft interlock bypassed (no-interlock), readback verification bypassed (no-readback), and fully ungated—on the live instance (mock drivers, seed 42, three repetitions per arm per released run, fresh fixture per repetition). Because the hard range is enforced structurally at the driver layer and cannot be bypassed, the ablation isolates exactly the two bypassable mechanisms. Fig. 4 reports the result; values were identical across repetitions (deterministic pipeline). Three findings: (i) bypassing the interlock collapses interception from 6/6 to 4/6 (identically in the no-interlock and ungated arms)—the window-class attacks execute while global-range attacks remain structurally rejected; (ii) the executed breaches are fully journaled (twelve out-of-window anchors per run: six breach writes plus six accepted edge probes, both bypass arms): *attribution is not prevention*; (iii) no arm adds latency resolvable at this tier (p50 110–141 ms, p95 150–169 ms across all arms). Boundary behavior follows each arm’s specification (9/9 probes = three probes $\times$ three repetitions; Wilson 95 % interval $[70, 100]$ %), false blocks are 0/3 in every arm, and the no-readback arm is behaviorally identical to Full on in-memory drivers (the mock readback echoes the command)—its verification value is exercised by the frozen-process injection above.

I. Case Study: LLM-Agent Closed-Loop Optimization (L4)

The benchmark of Sec. VI-G isolates the governed loop under a fixed controller; this subsection reports the complementary result—the same plant and objective closed by an LLM agent team—and it is a candid one. Four campaigns are released (first-party omp engine driving glm-5.3-flash in a lead-supervised team; manual-mode bindings; physics accelerated 6 $\times$; seed 42; plant and objective as in Sec. VI-G). In each campaign the team opened an optimization record, probed the plant with `daq_query`, and actuated through `dcw_control` over real protocol transports (Modbus TCP, OPC UA, MQTT, HTTP); the ledgers record 7, 7, 15 and 10 governed writes respectively, each attributable from journal anchors and the audit log, with the approval gate traversed on every write (approvals granted by the campaign’s scripted operator emulation under the campaign operator identity—an autonomous-campaign configuration, not a production posture).

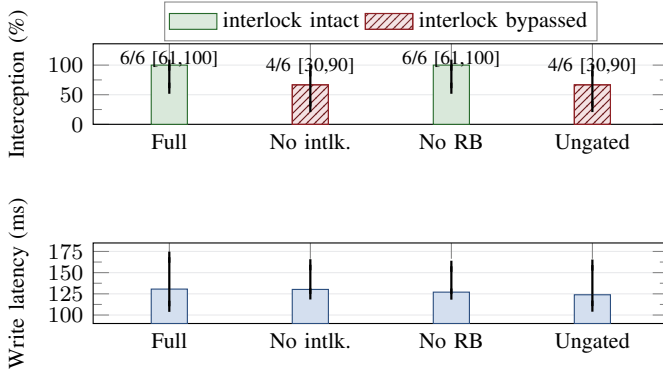


Fig. 4. E1a four-arm governance ablation (mock drivers, seed 42; three repetitions per run in each of three released runs, nine repetitions per arm, judge-class values identical across all nine). Top: interception per arm; error bars are Wilson 95 % intervals over the six seeded attacks, with the endpoints printed above each bar—structural rather than statistical at $n=6$, and drawn so the lower bound of a full-interception arm stays visible. Bottom: mean p50 write latency; whiskers span the minimum observed p50 to the maximum observed p95 across the nine repetitions. Per-arm aggregates recomputed from the released per-repetition CSVs by `bench/tools/fig4_analyze.py`.

The outcome does not flatter the agent. The agent objective reached $J_{\text{agent}} = 67.7, 31.2, 68.3, 68.5$ against $W^* = 89.894 - 0.75, 0.35, 0.76, 0.76$ of the offline optimum, median 0.76—and the thickness window $50 \pm 2 \mu\text{m}$ was *not* reached: three campaigns settled at a $55.33 \mu\text{m}$ window mean and the fourth produced no in-window samples, while the melt-pressure node failed its connection test on OPC UA in all four. Of 39 acceptance checks per campaign, 33–35 passed, the failures being the pressure node, the inspector audit, the thickness window, and the objective check.

We report this as a negative result with a design implication. It supplies the existence proof the framework needs—an LLM team drove real protocol transports through the governed path with every write reconstructable—while showing that on this objective the team does not match a two-step fixed controller (0.76 vs 0.97 of W^*). That is the propose/dispose division of labour working as intended: the agent explores and proposes, and the deterministic path is what converts proposals into converged actuation. It also sets the bar the released agent-in-the-loop campaigns (E1b, E2, E3) must clear before an LLM supervisor can be claimed to add value over that path.

J. Reproducibility and Threats to Validity

Reproducibility is separated into what is exactly reproducible and what is not. Re-running the whole pipeline twice with an identical harness (source hash `204fc9ee`), seed, and commit yields *bit-identical* structural results: 55/55 pipeline checks pass in both runs; every seeded attack is rejected with zero false blocks on *both* plant scenarios (9/9 on the second); all four per-line loops converge; the portability aggregates and backstop drill outcomes are identical; and $W^* = 89.894$. What varies is what the abstraction predicts: governed-write latency is environment-dependent, and the closed-loop objective—computed from *sampled* process variables—lands in $J/J^* \in [0.966, 0.973]$ across campaign repetitions. Every run appends

to a never-overwritten timestamped directory; the archive for this revision holds more than 90 run records across tiers, each carrying its seed, harness hash, and commit.

Threats to validity. **Construct:** success oracles are database- and readback-defined, mitigating self-reported success bias. **Internal:** temperature-0, pinned-version LLM arms reduce but do not eliminate variance; hence mock-arm matrices with LLM spot-checks and nonparametric tests. **External:** all measured layers are simulation-tier (two seeded plant scenarios, bounded by the ODE twin’s fidelity); public-dataset replay (TEP, SWaT) is designed but unmeasured; the four LLM campaigns share one seed and carry no cross-seed variance estimate. Results are layer-labeled (Table I) and we claim no transfer beyond L1–L3–L4 without per-layer evidence. **Conclusion:** the closed-loop result is reported over a fixed, pre-declared controller and three plant seeds, and the LLM campaign is reported as a measured negative rather than as a headline.

VII. DISCUSSION AND LIMITATIONS

A. Scope and Limits

The pipeline constrains the *action language* of agents; it does not make agents good at choosing actions. Proposal quality still derives from LLM competence, context quality, and tool ergonomics: bounded mechanisms (interlock, readback, chain limit) absorb bad proposals, while the unbounded statistical components (LLM decisions, with deterministic rule fallbacks) sit in a tier whose worst case is the rules’ performance. The invariants borrow IEC 61511’s separation of basic control from safety functions [27] but remain engineering properties, not certified safety functions: the by-construction guarantee holds within the sanctioned path of Sec. IV-D, and AGENTWORKSHOP does not replace certified instrumentation. The completed evidence covers the artifact audit, live-instance invariant verification, the backstop’s dynamically exercised rollback (chain limit $K=2$ still specified), real-protocol scenarios on two plant scenarios, the pipeline-tier ablation, and one plant domain at L1/L4; the agent-in-the-loop campaigns (E1b–E9) remain specified—AW-INDUSTRIALBENCH measures *governed supervisory control*, not open-ended plant engineering. Three lessons may transfer: single-entry writes are the keystone; judgment/execution separation pays twice; deterministic rehearsal with an offline-optimum oracle surfaced defects before hardware attachment.

VIII. CONCLUSION

We presented AGENTWORKSHOP, an integration framework embedding LLM multi-agent teams natively into the supervisory fabric of protocol-heterogeneous production lines—one process, one data domain, physical semantics instead of register addresses, team-level node binding, configuration-only commissioning—with AW-INDUSTRIALBENCH, a reproducible evaluation protocol. Every agent-issued write is forced through a single governed pipeline defining three safety invariants over the accepted-action language, independent of engine. A governed closed-loop benchmark on a seeded cast-film line recovers 97.0–97.2 % of the offline grid optimum in two iterations and at most two governed writes per seed ($J/J^* \in$

[0.966, 0.973] over repetitions); the backstop's rollback is exercised end-to-end on a live breach; and four LLM-agent campaigns actuate the same loop over real transports with every write attributed, reaching a median 76 % of the optimum—a measured shortfall that motivates the propose/dispose split rather than being papered over. The remaining campaigns of AW-INDUSTRIALBENCH are released as executable protocol designs; the platform source (PolyForm Noncommercial), the benchmark harness, the seeded simulator, and the run archives accompany the paper.

REFERENCES

- [1] M. Raza *et al.*, “Industrial applications of large language models,” *Scientific Reports*, vol. 15, 2025, art. no. 13755.
- [2] S. Colabianchi, F. Costantino, and N. Sabetta, “Assessment of a large language model based digital intelligent assistant in assembly manufacturing,” *Computers in Industry*, vol. 162, 2024, art. no. 104129.
- [3] J. Zhang, H. Wang, Z. Jia, J. Dong, and L. Ren, “CoMA-IKG: LLM-driven multiagent framework for automated construction of industrial knowledge graph,” *IEEE Transactions on Industrial Informatics*, vol. 22, no. 6, pp. 5553–5564, 2026.
- [4] Z. Liu, R. Zeng, D. Wang, G. Peng, J. Wang, Q. Liu, P. Liu, and W. Wang, “Agents4PLC: Automating closed-loop PLC code generation and verification in industrial control systems using LLM-based agents,” *IEEE Transactions on Software Engineering*, vol. 52, 2026, also arXiv:2410.14209.
- [5] Ren *et al.*, “MetaIndux-PLC: A control logic-guided LLM for PLC code generation in industrial control systems,” *Applied Soft Computing*, vol. 184, 2025, art. no. 113673.
- [6] Y. Xia, D. Dittler, N. Jazdi, H. Chen, and M. Weyrich, “LLM experiments with simulation: Large language model multi-agent system for simulation model parametrization in digital twins,” in *Proc. IEEE ETFA*, 2024, arXiv:2405.18092.
- [7] M. S. Gill, J. Vyas, A. Markaj, F. Gehlhoff, and M. Mercangöz, “Leveraging LLM agents and digital twins for fault handling in process plants,” in *Proc. IEEE ETFA*, 2025, arXiv:2505.02076.
- [8] B. Hofmann, N. Piechulek, S. Kreitlein, J. Franke, and P. Bruendl, “Beyond touch-based human-machine interface: Control your machines in natural language by utilizing large language models and OPC UA,” *arXiv preprint arXiv:2510.11300*, 2025.
- [9] J. Vyas, M. S. Gill, A. Markaj, F. Gehlhoff, and M. Mercangöz, “From detection to action: Using LLM agents for fault-tolerant control,” *arXiv preprint arXiv:2606.28011*, 2026.
- [10] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes, “Autonomous chemical research with large language models,” *Nature*, vol. 624, pp. 570–578, 2023.
- [11] S. H. Vemprala, R. Bonatti, A. Bucker, and A. Kapoor, “Chatgpt for robotics: Design principles and model abilities,” 2023.
- [12] International Society of Automation, “Ansi/isa-18.2: Management of alarm systems for the process industries (iec 62682),” Standard, 2016.
- [13] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, “ τ -bench: A benchmark for tool-agent-user interaction in real-world domains,” *arXiv preprint arXiv:2406.12045*, 2024.
- [14] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, “Control barrier function based quadratic programs for safety critical systems,” *IEEE Transactions on Automatic Control*, vol. 62, no. 8, pp. 3861–3876, 2017.
- [15] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, “Safe reinforcement learning via shielding,” in *Proc. AAAI Conference on Artificial Intelligence*, 2018.
- [16] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, “A model for types and levels of human interaction with automation,” *IEEE Transactions on Systems, Man, and Cybernetics—Part A: Systems and Humans*, vol. 30, no. 3, pp. 286–297, 2000.
- [17] International Society of Automation, “Ansi/isa-95.00.01: Enterprise-control system integration,” Standard, 2010.
- [18] —, “Ansi/isa-88.01: Batch control models and terminology,” Standard, 1995.
- [19] International Electrotechnical Commission, “Iec 63278 (series): Asset administration shell for industrial applications,” International Standard, 2023.
- [20] X. Liu *et al.*, “AgentBench: Evaluating LLMs as agents,” in *Proc. ICLR*, 2024, arXiv:2308.03688.
- [21] J. J. Downs and E. F. Vogel, “A plant-wide industrial process control problem,” *Computers & Chemical Engineering*, vol. 17, no. 3, pp. 245–255, 1993.
- [22] J. Goh, S. Adep, K. N. Junejo, and A. Mathur, “A dataset to support research in the design of secure water treatment systems,” in *Proc. 11th Int. Conf. Critical Information Infrastructures Security (CRITIS)*, 2017, pp. 88–99.
- [23] Q. Wu, G. Bansal, J. Zhang, T. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. Awadallah, R. White, D. Burger, and C. Wang, “AutoGen: Enabling next-gen LLM applications via multi-agent conversation,” *arXiv preprint arXiv:2308.08155*, 2023.
- [24] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *Proc. ICLR*, 2023.
- [25] Anthropic, “Model context protocol,” <https://modelcontextprotocol.io>, 2024.
- [26] International Electrotechnical Commission, “IEC 62443: Security of industrial automation and control systems,” International Standard, 2018.
- [27] —, “IEC 61511: Functional safety—safety instrumented systems for the process industry sector,” International Standard, 2016.
- [28] —, “IEC 62541: OPC unified architecture,” International Standard, 2020.
- [29] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, “Reciprocal rank fusion outperforms Condorcet and individual rank learning methods,” in *Proc. ACM SIGIR*, 2009, pp. 758–759.
- [30] J. Carbonell and J. Goldstein, “The use of MMR, diversity-based reranking for reordering documents and producing summaries,” in *Proc. ACM SIGIR*, 1998, pp. 335–336.
- [31] Eclipse Foundation, “Eclipse ditto: Digital twin framework,” 2024.
- [32] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection,” *arXiv preprint arXiv:2302.12173*, 2023.